

작업 정리 — 2026년 7월 30일 ~ 8월 1일

이 문서는 누구를 위한 것인가 이 프로젝트를 처음 보는 사람도 "무슨 일이 있었고, 왜 그렇게 했고, 그래서 뭐가 달라졌고, 앞으로 뭘 할 건지"를 따라올 수 있게 쓴 글이다. 전문용어가 나올 때마다 그 자리에서 비유로 풀어 설명한다. 맨 뒤에 용어 사전도 있다.

기술적 세부사항과 미결 사항의 공식 기록은 [handoff.md](#)에 있다. 이 문서는 그것을 "사람이 읽는 이야기"로 옮긴 것이다. 숫자가 어긋나면 [handoff.md](#)가 기준이다.

목차

1. 이 프로젝트가 뭘 하는 건지
2. 핵심 개념 6개
3. 이번 작업의 출발점
4. 발견 1 — 요리사에게 잘못 가르치고 있었다
5. 발견 2 — 모의고사가 부정행위였다
6. 내가 낸 버그 — 정직하게 기록한다
7. 발견 3 — 가장 큰 것: AI가 학습을 포기하고 있었다
8. 요청받은 실험 — 커널에 "과거"를 넣어보기
9. 숫자로 본 최종 결과
10. 지금 어디까지 왔나
11. 앞으로 갈 방향
12. 이번 일에서 배운 것
13. 용어 사전

1. 이 프로젝트가 뭘 하는 건지

1-1. 한 문장으로

자동차가 정해진 길을 정확히 따라가도록 핸들을 자동으로 돌려주는 시스템을 만들고, 그 시스템이 "나는 지금 얼마나 확신하는가"를 정직하게 말할 수 있는지 검증하는 것.

두 번째 부분이 이 프로젝트의 진짜 주제다. 잘 따라가는 것보다 **"내가 틀릴 수도 있다는 걸 아는가"**가 더 중요하다.

1-2. 왜 두 번째가 더 중요한가

자율주행차가 커브를 돌고 있다고 하자. 두 가지 상황을 비교해 보자.

- **상황 A:** 차가 경로에서 10cm 벗어났다. 그런데 시스템은 "나는 정확하다"고 믿고 있다.
- **상황 B:** 차가 경로에서 10cm 벗어났다. 시스템은 "지금 여긴 처음 보는 상황이라 30cm까지 벗어날 수 있다"고 말한다.

B가 훨씬 안전하다. 왜냐하면 B는 "속도를 줄여라", "사람에게 넘겨라" 같은 다음 행동을 할 수 있게 해주기 때문이다. A는 사고가 날 때까지 자기가 틀린 줄 모른다.

의사로 비유하면, "이 검사 결과로는 확실히 모르겠으니 정밀검사를 받으십시오"라고 말하는 의사가, 모르면서 자신 있게 진단하는 의사보다 낫다. **자기 무지를 아는 것이 능력이다.**

이 프로젝트는 그 "자기 무지를 아는 능력"을 두 가지 방법으로 만들어 보고, 어느 쪽이 더 정직한지 비교한다.

1-3. 자동차를 어떻게 다루는가 — "모델"

컴퓨터가 핸들을 얼마나 돌릴지 계산하려면, **핸들을 이만큼 돌리면 차가 이렇게 움직인다**는 예측이 있어야 한다. 이 예측 공식을 ****모델(model)****이라고 부른다.

비유 — 모델은 지도다. 지도는 실제 땅이 아니다. 산을 삼각형으로, 강을 파란 선으로 단순하게 그린 것이다. 그 래도 길 찾는 데는 충분하다. 모델도 마찬가지다. 실제 자동차는 타이어 고무가 늘어나고, 서스펜션이 출렁이고, 노면이 울퉁불퉁하지만, 모델은 그걸 다 무시하고 간단한 수식 몇 개로 줄인 것이다.

이 프로젝트가 쓰는 모델은 ****선형 자전거 모델(linear bicycle model)****이다.

비유 — 자동차를 자전거로 본다. 자동차는 바퀴가 4개지만, 좌우 바퀴를 하나로 합쳐서 **앞바퀴 1개 + 뒷바퀴 1개**인 자전거처럼 취급한다. 실제로 자동차 제어 분야에서 100년 가까이 쓰인 표준 근사다. 계산이 훨씬 간단해지고, 웬만한 상황에서는 충분히 정확하다.

"선형(linear)"은 **입력을 2배 하면 결과도 정확히 2배가 된다**는 뜻이다. 핸들을 2배 돌리면 회전도 정확히 2배가 된다는 가정이다.

1-4. 모델이 틀리는 지점 — 타이어

문제는 **이 가정이 실제로는 틀린다**는 것이다. 특히 타이어에서.

비유 — 고무줄과 타이어. 고무줄을 살살 당기면 당긴 만큼 늘어난다(선형). 그런데 세게 당기면? 어느 순간부터 힘을 더 줘도 잘 안 늘어나고, 더 세게 하면 끊어진다. 타이어도 똑같다. 살살 돌릴 때는 "핸들 2배 = 회전 2배"가 맞지만, 급커브에서 세게 돌리면 타이어가 미끄러지기 시작하면서 이 비례관계가 무너진다.

그래서 **빠르게 코너를 돌수록 모델의 예측이 실제와 벌어진다**. 이 프로젝트에서 "가속도 a_y 를 4에서 6으로 올린다"는 말은 **더 세게 코너를 돈다**는 뜻이고, 그러면 모델이 더 많이 틀린다.

1-5. 틀린 만큼을 뭐라고 부르나 — "잔차"

잔차(residual) = 모델이 예측한 값과 실제로 일어난 일의 차이.

비유 — 일기예보의 오차. 예보가 "내일 25도"라고 했는데 실제로 28도였다면, 잔차는 +3도다. 잔차가 0이면 예보가 완벽했다는 뜻이고, 잔차가 크면 예보 모델에 뭔가 빠졌다는 뜻이다.

이 프로젝트의 핵심 아이디어는 이것이다:

모델을 더 복잡하게 만드는 대신, 간단한 모델을 그대로 쓰고 "얼마나 틀리는지 (잔차)"를 따로 학습해서 더해준다.

지도 비유로 돌아가면, 낡은 지도를 새로 그리는 대신 **"이 지도는 여기서 500m쯤 틀립니다"**라고 적힌 포스트잇을 붙이는 것과 같다.

1-6. 두 가지 보정 방법 — 이 프로젝트의 비교 대상

잔차를 메우는 방법이 두 가지 있고, 이 둘을 비교하는 게 프로젝트의 Part 1이다.

① 칼만 필터 (KF, Kalman Filter) — 시간의 흐름을 따라가는 방법

비유 — 어제 늦었으니 오늘도 늦겠지. 친구가 최근 3번 연속 10분씩 늦었다면, 오늘도 10분쯤 늦을 거라고 예상한다. 칼만 필터는 이런 식으로 "**방금 전까지 이만큼 틀렸으니 지금도 비슷하게 틀리겠지**" 하고 보정한다. 아폴로 달 착륙선부터 스마트폰 GPS까지 안 쓰이는 데가 없는, 60년 넘게 검증된 방법이다.

② 가우시안 프로세스 (GP, Gaussian Process) — 상황을 보고 판단하는 방법

비유 — 경력 30년 정비사. "이 차종이 이 속도로 이 각도로 코너를 돌 때는, 보통 이만큼 밀립니다." 시간 순서가 아니라 **지금의 상황(속도, 회전율, 핸들 각도)**을 보고 판단한다. 예전에 비슷한 상황을 많이 겪어왔기 때문에 가능한 일이다.

GP가 특별한 이유: GP는 답만 주는 게 아니라 "**제가 이 답에 얼마나 자신 있는지**" 를 숫자로 같이 준다.

비유 — 정비사의 자신감. "이런 상황은 수백 번 봤어요. 확실히 5cm 밀립니다." (자신 있음) "이런 건 처음 보는 데... 5cm쯤? 근데 20cm일 수도 있어요." (자신 없음)

GP는 **학습한 데이터가 많은 상황에서는 자신 있게, 처음 보는 상황에서는 자신 없게** 답하도록 수학적으로 설계돼 있다. 이게 프로젝트가 GP를 쓰는 이유다.

2. 핵심 개념 6개 (이것만 알면 나머지가 읽힌다)

이 6개를 이해하면 아래 내용이 전부 읽힌다. 천천히 읽어도 좋다.

개념 1: MPC — "몇 수 앞을 내다보고 핸들을 돌린다"

****MPC(Model Predictive Control, 모델 예측 제어)****는 이 프로젝트가 쓰는 제어 방식이다.

비유 — 체스와 운전. 초보는 지금 눈앞의 수만 본다. 고수는 "내가 이렇게 두면 상대가 저렇게 두고, 그럼 내가 이렇게..." 하고 몇 수 앞을 계산한 뒤 **지금의 한 수**를 결정한다.

MPC가 정확히 이렇게 한다. **앞으로 1초 동안** 핸들을 어떻게 돌릴지 전부 계산해 놓고, 그중 **맨 첫 번째 값만 실제로 실행**한다. 그리고 0.02초 뒤에 처음부터 다시 계산한다. 계획은 길게, 실행은 한 걸음씩.

이 프로젝트에서는 앞으로 **50 스텝(= 1초)**을 내다본다. 계산을 1초에 50번 반복한다. 즉 **한 번 계산에 0.02초(20밀리초) 안에 끝내야 한다**. 이 시간 제약이 뒤에서 "M=4000은 못 쓴다" 같은 판단의 근거가 된다.

개념 2: 캘리브레이션 — "자신감이 정직한가"

이 프로젝트의 **주 평가 기준**이다. 정확도가 아니라 **자신감의 정직성**을 잴다.

비유 — 강수확률. 기상청이 "비 올 확률 70%"라고 한 날들만 모아보자. 그중 실제로 비가 온 날이 **70%쯤**이면 그 예보는 **캘리브레이션이 잘 된** 것이다.

- 70%라고 한 날 중 30%만 비가 왔다면 → **과신(overconfident)**. 너무 자신만만했다.
- 70%라고 한 날 중 95%나 비가 왔다면 → **과소신뢰**. 쓸데없이 겁먹었다.

둘 다 문제지만 **과신이 훨씬 위험하다**. 자율주행에서 과신은 "괜찮다"고 말하다가 사고 나는 것이고, 과소신뢰는 "위험하다"고 말하다가 불필요하게 멈추는 것이다.

캘리브레이션을 재는 숫자가 3개 있다. **이 3개는 이 문서 전체에서 계속 나온다**.

지표	뜻	좋은 값	읽는 법
z_std	실제 오차가 예측한 오차범위의 몇 배인가	1.0	1보다 크면 과신 (실제로 더 많이 틀림), 작으면 과소신뢰
95% 커버	"95% 확률로 이 안에 있다"고 한 범위에 실제로 들어간 비율	95%	95%보다 낮으면 과신
NLPD	예측 분포 전체의 품질 점수	낮을수록 좋음	음수로 갈수록 좋다. -6이 -5보다 좋다

z_std를 좀 더 풀면: 시스템이 "오차는 $\pm 5\text{cm}$ 쯤일 겁니다"라고 했는데 실제로는 $\pm 11\text{cm}$ 틀렸다면 $z_std \approx 2.2$ 다. **자기가 말한 것보다 2배 넘게 틀렸다는 뜻이다.** 이 숫자가 이 문서에서 가장 중요하다.

NLPD를 좀 더 풀면: Negative Log Predictive Density의 약자다. "예측한 확률분포가 실제 일어난 일에 얼마나 높은 확률을 줬는가"를 점수화한 것이다. **비유:** 시험에서 객관식이 아니라 "각 보기에 확률을 배분하라"는 문제라고 하자. 정답에 90% 준 사람이, 정답에 30% 준 사람보다 점수가 좋다. 대신 정답에 5%밖에 안 줬는데 그게 정답이면 크게 감점된다. NLPD는 그 감점의 총합이고, **낮을수록 좋다.** 평균 정확도와 자신감 크기를 **한꺼번에** 보는 종합 점수라고 보면 된다.

개념 3: in-distribution vs OOD — "배운 것 vs 안 배운 것"

in-distribution(분포 안) = 학습할 때 봤던 것과 비슷한 상황 **OOD(Out-Of-Distribution, 분포 밖)** = 학습 때 못 봤던 낯선 상황

비유 — 수학 문제집. 문제집을 열심히 푼 학생이 있다.

- **in-distribution:** 문제집에 있던 것과 **비슷한 유형**이 시험에 나옴 → 잘 푼다.
- **OOD:** 완전히 **처음 보는 유형**이 나옴 → 헤맨다.

여기서 중요한 건, **잘 못 푸는 것 자체는 죄가 아니라는 것이다.** 문제는 **"모르는 걸 아는가"**다. 처음 보는 문제를 틀리면서도 "확실히 맞았다"고 우기면 그게 진짜 문제다. GP한테 우리가 원하는 건 **"이건 처음 보는 유형이라 자신 없다"**고 말하는 것이다.

이 프로젝트의 4개 평가 시나리오가 정확히 이 축으로 짜여 있다:

#	이름	무엇이 낯선가	난이도
1	lpf_ay4	아무것도 — 배운 코스, 배운 속도	in-distribution (진단용)
2	lpf_ay6	속도가 더 빠름 (강도 외삽)	중간
3	rtf_ay4	코스가 다름 (형상 외삽)	중간
4	rtf_ay6	코스도 다르고 더 빠름 (둘 다)	가장 어려움

- **lpf** = loop10이라는 학습용 코스 + 센서 잡음 + 필터
- **rtf** = racetrack이라는 다른 코스 + 센서 잡음 + 필터
- **ay4 / ay6** = 코너링 가속도 4 또는 6 (클수록 빠르고 거칠게 돈다)

1번은 "일반화를 잘한다"의 증거가 될 수 없다. 학습한 코스에서 그대로 시험 본 것이기 때문이다. 1번은 오직 **"학습한 것조차 제대로 못 하고 있는가"**를 확인하는 진단용이다.

개념 4: 개루프 vs 페루프 — "모의고사 vs 실전"

개루프(open-loop) = GP의 예측이 자동차의 움직임에 **영향을 주지 않는** 상태에서 채점 **페루프(closed-loop)** = GP가 실제로 핸들 조작에 **관여하면서** 달리는 상태에서 채점

비유 — 축구 해설과 축구 선수.

- **개루프**: 이미 끝난 경기 영상을 보며 "여기서 왼쪽으로 패스했어야 한다"고 평가. 내 말이 경기 결과를 바꾸지 않는다. 채점이 쉽다.
- **페루프**: 내가 직접 뛴다. 내 판단이 다음 상황을 **바꾼다**. 내가 왼쪽으로 가면 상대 수비도 왼쪽으로 오고, 그래서 다음 순간의 상황 자체가 달라진다.

페루프가 훨씬 어렵다. **내 실수가 다음 상황을 만들기 때문이다**. 그리고 실제로 쓰이는 건 페루프다.

이 구분이 이번 작업의 출발점이었다. 자세한 건 3장.

개념 5: 커널과 차원 — "무엇을 보고 비슷하다고 판단하는가"

GP는 "지금 상황이 예전에 본 어떤 상황과 비슷한가"를 따져서 답을 낸다. 이때 **무엇을 근거로 '비슷하다'고 판단할지** 정해주는 게 ****커널(kernel)****이다.

비유 — 닮음을 재는 자. 사람 둘이 "비슷한가"를 판단할 때, 키만 보면 다른 결론이 나오고, 키+몸무게+나이를 보면 또 다른 결론이 나온다. 커널은 *****어떤 항목들을 어떤 비중으로 볼지 정한 자*****다.

이 프로젝트에서 GP가 보는 항목은 원래 3개였다:

```
z = [ 횡속도(v_y), 회전율(gamma), 핸들각(delta) ]
```

이 항목 개수를 ****차원(dimension)****이라고 한다. 여기서는 3차원.

차원을 늘리면 좋을 것 같지만, 공짜가 아니다.

비유 — 차원의 저주(curse of dimensionality). 1미터짜리 막대 위에 점 10개를 뿌리면 10cm마다 하나씩, 촘촘하다. 1m × 1m 정사각형에 10개를 뿌리면? 듥성듥성해진다. 1m × 1m × 1m 정육면체에 10개면? 거의 텅 빈다.

차원이 늘어날수록 같은 데이터로 채워야 할 공간이 폭발적으로 커진다. 그래서 차원을 늘리면 "여긴 데이터가 있었나?"라는 GP의 판단이 흐려지고, 이 프로젝트의 핵심 주장("데이터 없는 곳에서 자신 없어 한다")이 약해진다.

프로젝트 규칙 파일([.claude/rules/gp-residual.md](#))에 *****차원 최소 유지가 설계 목표다**. 입력 차원을 늘리기 전에 반드시 물어라"***라고 못박혀 있는 이유가 이것이다.

개념 6: 하이퍼파라미터 학습 — "AI가 스스로 정하는 설정값"

GP에는 사람이 직접 정하지 않고 **데이터를 보고 자동으로 정하는 설정값**들이 있다. 이것 ****하이퍼파라미터(hyperparameter)****라고 한다. 중요한 게 3개다.

이름	뜻	비유
lengthscale	"얼마나 떨어지면 다른 상황으로 볼지"	시야 범위. 짧으면 근시안(바로 옆만 비슷하다고 봄), 길면 원시안
sigma_f	신호의 크기 — "설명할 수 있는 변화가 얼마나 큰가"	목소리 크기
sigma_n	잡음의 크기 — "설명 못 하는 무작위 변동이 얼마나 큰가"	주변 소음 크기

sigma_f와 sigma_n의 관계가 핵심이다. 데이터에 나타난 변동을 GP가 두 몫으로 나눈다: "내가 설명할 수 있는 부분(sigma_f)"과 "그냥 무작위 잡음(sigma_n)".

비유 — 성적 변동 해석하기. 시험 점수가 오르락내리락한다. 이것을 어떻게 해석할까?

- "공부 시간에 따라 달라진다" → 설명 가능한 신호 (sigma_f가 큼)
- "그날 컨디션 운이다" → 설명 불가능한 잡음 (sigma_n이 큼)

GP는 데이터를 보고 이 비율을 스스로 정한다. 그 방법이 **Type-II ML**이다.

Type-II ML (Type-II Maximum Likelihood, 2종 최대우도법) 이름은 거창하지만 아이디어는 단순하다. "내가 관찰한 데이터가 나올 확률이 가장 높아지는 설정값을 고른다."

비유 — 범인 찾기. "이 발자국 크기, 이 보폭이라면 키 175cm인 사람일 확률이 제일 높다"고 추정하는 것과 같다. 증거를 가장 잘 설명하는 가설을 고르는 것.

이때 "얼마나 잘 설명하는가"를 재는 점수가 **NLML**이다. **낮을수록 좋다.** (7장에서 이 숫자가 결정적으로 쓰인다.)

3. 이번 작업의 출발점 — 하나의 이상한 숫자

3-1. 먼저: 밀린 작업 커밋 정리

작업 시작 시점에 **25개 파일이 수정됐는데 하나도 저장(커밋)되지 않은 상태였다.** 1,065줄 분량이 저장 안 된 채 쌓여 있었다.

커밋(commit)이란? 코드의 "저장 지점"을 만드는 것. 게임의 세이브 포인트와 같다. 여러 개를 만들어 두면 "3번째 세이브로 돌아가자" 같은 게 가능해진다. 저장을 안 하면? 뭔가 잘못됐을 때 **어디서부터 잘못됐는지 되짚을 수가 없다.**

이것을 논리적으로 5개 커밋으로 나눠 저장했다. 테스트 77개가 전부 통과하는 걸 확인한 뒤에 했다.

커밋	내용
08c899f	칼만 필터 재튜닝 + NIS 진단 추가
6631535	페루프 N바퀴 주행 기능 + GP 학습 데이터 출처 일원화
ad2806f	GP 예측분산 버그 수정 + 딕셔너리 크기 M=2000
4d7c816	4케이스 실험 체계 신설

커밋	내용
41fdd2c	기록 문서 갱신

3-2. 이상한 숫자 하나

기록을 보니 이런 게 있었다. 같은 GP인데 평가 방식만 다른데 점수가 단판이었다.

개루프(모의고사)로 채점:

$z_std = 1.241$

→ 거의 정직함

페루프(실전)로 채점:

$z_std = 2.192$

→ 심하게 과신

같은 학생이 모의고사는 1등급인데 수능은 5등급인 셈이다. 뭔가 있다.

이 격차가 왜 중요하냐면, 이걸 모르면 "GP로 이 문제를 풀 수 있다"는 주장의 범위를 정할 수 없기 때문이다. 모의고사 점수를 믿어야 하나, 수능 점수를 믿어야 하나?

3-3. 코드를 뜯어서 찾은 차이점들

두 채점 방식이 정확히 뭐가 다른지 코드를 읽어서 목록을 만들었다.

	개루프 채점	페루프 채점
데이터 출처	학습에 쓴 바로 그 주행 기록	새로 달린 주행
운전한 컨트롤러	GP 없이(mpc_only) 달림	GP가 관여하며 달림
GP가 궤적에 영향?	없음	있음
핸들각 특징 시점	δ_k (그 순간 밟은 핸들)	δ_{k-1} (직전 핸들)

마지막 줄을 발견한 순간이 이번 작업의 출발점이었다.

4. 발견 1 — 요리사에게 잘못 가르치고 있었다

4-1. 무엇이 문제였나

GP가 보는 3개 항목 중 하나가 **핸들각(delta)**이다. 그런데:

- **학습할 때:** "이 순간 핸들을 δ_k 만큼 돌렸더니 이만큼 틀리더라"로 가르쳤다.
- **실제 사용할 때:** GP한테 물어보는 시점이 핸들을 얼마나 돌릴지 계산하기 전이다. 그래서 δ_k 를 알 수가 없고, 어쩔 수 없이 **직전 값 δ_{k-1} **을 넣어서 물어본다.

즉 가르친 것과 물어보는 것이 달랐다.

비유 — 요리사 훈련의 오류. 요리 학원에서 이렇게 가르쳤다고 하자. "소금을 5g 넣으면 이런 맛이 난다."

그런데 실전 주방에서는 이렇게 묻는다. "아직 소금을 안 넣었는데, 지금 맛이 어떨 것 같아?"

배운 조건과 질문받는 조건이 다르다. 요리사는 최선을 다해 답하겠지만, 자기가 배운 것과 다른 질문을 받고 있다는 사실조차 모른다. 그래서 틀리면서도 자신 있게 답한다. 이게 정확히 과신의 메커니즘이다.

4-2. 왜 이런 게 생겼나 — "인과성"

여기서 **인과성(causality)** 개념이 나온다.

인과성 = "결과가 나오기 전에 원인을 알 수는 없다"는 시간의 순서.

비유: 로또 번호를 보고 로또를 살 수는 없다. 주가가 오른 걸 확인하고 어제 주식을 살 수는 없다. 당연한 말 같지만, 데이터를 다룰 때는 **미래 정보가 슬쩍 섞여 들어가는 일이 놀랄 만큼 자주 일어난다.** 이걸 데이터 누수(leakage)라고 한다.

우리 경우가 딱 이거였다. **δ_k 는 GP한테 물어보는 시점 기준으로 "미래 정보"였다.** 학습할 때만 그걸 쓸 수 있었고, 실전에서는 못 쓴다.

4-3. 어떻게 고쳤나

방향은 하나뿐이었다. **실전에서 δ_k 를 쓸 방법이 없으니, 학습을 실전에 맞춘다.**

- 시뮬레이터가 `delta_prev`(= 실제로 GP에 넘긴 바로 그 값)를 기록하게 했다.
- 학습이 그 값을 쓰게 했다.
- **정답(잔차)의 정의는 건드리지 않았다.** 문제지만 바꾸고 답안지는 그대로.

4-4. 다시는 안 생기게 막는 장치 — "회귀 테스트"

회귀 테스트(regression test)란? 한 번 고친 버그가 나중에 슬그머니 되살아나는 것을 막는 자동 검사다.

비유 — 문단속 확인 알람. 문 잠그는 걸 한 번 깜빡해서 사고가 났다면, 다음부터는 "문 잠갔나?" 하고 자동으로 확인해 주는 장치를 다는 것이다. 사람의 기억력에 의존하지 않는다.

`test_train_deploy_feature_match`라는 테스트를 만들었다. 이 테스트는 **"학습이 쓰는 값"과 "실전이 쓰는 값"이 매 순간 똑같은지** 자동으로 대조한다. 누군가 나중에 한쪽만 고치면 즉시 실패한다.

참고: 이 프로젝트는 이미 **같은 종류의 실수를 한 번 겪었다.** 2026년 7월 31일에 "센서 잡음이 낀 값으로 실전을 돌리면서 학습은 깨끗한 참값으로 했던" 불일치를 고쳤다. **이번 건 그 형제 버그였다** — 상태(x)는 고쳤는데 핸들 각(delta)은 안 고쳐져 있었던 것이다.

5. 발견 2 — 모의고사가 부정행위였다

5-1. 홀드아웃이란

홀드아웃(holdout) = 가진 데이터 중 일부를 일부러 학습에서 빼놓고, 나중에 채점용으로만 쓰는 것.

비유 — 문제집 뒷장 남겨두기. 문제집 100문제 중 80개로 공부하고 20개는 안 보고 남겨둔다. 그 20개로 시험을 보면 **진짜 실력**을 알 수 있다. 100개 다 풀어보고 그중에서 시험을 내면 그냥 외운 걸 확인하는 것뿐이다.

5-2. 무엇이 잘못됐나

이 프로젝트의 학습 데이터는 **같은 코스를 10바퀴 돈 기록**(52,665 스텝)이다. 그중 2,000개를 뽑아 학습하고, **나머지 중에서 아무거나 뽑아 채점**하고 있었다.

문제는 **뽑는 방식**이었다. 2,000개를 52,665개에서 균일한 간격으로 뽑으니 **26 스텝(0.53초)마다 하나씩** 뽑힌 셈이 된다. 그러면 채점용 점은 ****항상 학습용 점 바로 옆(0.26초 이내)****에 있게 된다.

비유 — 시험 범위에서 문제 뽑기. 선생님이 "1번, 3번, 5번, 7번... 홀수 문제로 공부해라" 하고, 시험에는 짝수 문제를 냈다고 하자. 공정해 보이는가? **전혀 아니다.** 2번 문제는 1번, 3번과 거의 똑같은 문제일 것이다. 학생은 **진짜로 이해하지 않고 옆 문제를 보고 찍어도 맞힌다.**

초당 50번 기록되는 자동차 데이터에서 0.26초 차이는, 사실상 **거의 같은 순간**이다.

그리고 이건 단순한 실수가 아니라 **프로젝트 규칙 위반**이었다. [gp-residual.md](#)에 이렇게 적혀 있다:

"학습 궤적과 평가 궤적을 분리한다. 같은 주행 로그에서 학습·평가를 동시에 하지 마라."

규칙이 이미 금지하고 있던 걸 하고 있었던 것이다.

5-3. 어떻게 고쳤나 — "랩 블록 분할"

바퀴 단위로 자르기로 바꿨다.

1~9바퀴 → 학습용 (47,390개)
10바퀴째 → 채점용 (5,265개)

이러면 채점용 구간의 어떤 점도 학습 구간과 시간적으로 붙어 있지 않다. 같은 코스를 다시 도는 것이라 **"상황의 종류는 같고 실제 궤적만 다른"**, 딱 재고 싶은 조건이 된다.

비유 — 같은 문제집, 다른 회차. 1~9회차로 공부하고 10회차로 시험 보는 것. 유형은 같지만 문제 자체는 처음 보는 것들이다.

6. 내가 낸 버그 — 정직하게 기록한다

5장의 수정을 하면서 **내가 새 버그를 만들었다.** 이 프로젝트의 규칙에 *****물리적으로 이상한 결과가 나오면 숫자를 포장하지 말고 그대로 보고하라*****는 조항이 있으므로, 그대로 적는다.

6-1. 첫 측정 결과가 이상했다

랩 블록 분할로 처음 재봤더니 이런 숫자가 나왔다:

$z_std \approx 0.04 \sim 0.15$ (좋은 값은 1.0)
95% 커버 = 100.0% (좋은 값은 95%)

이건 과신이 아니라 **극단적인 과소신뢰**다. 예측 오차범위가 실제보다 **10~25배 크다**는 뜻이다. 그리고 6개 변형이 **전부** 같은 방향으로 극단값이 나왔다.

여러 변형이 한꺼번에 같은 극단값을 보이면 모델 문제가 아니라 측정 장치 문제다. 바로 앞 측정에서 같은 지표가 1.241이었는데 분할 방식만 바뀌 0.08이 되는 건 설명이 되지 않았다.

6-2. 원인 — 4.7미터 때문에

바퀴 수를 세는 코드가 이랬다:

```
바퀴 수 = floor(전체거리 / 한바퀴거리) + 1
```

한 바퀴가 2,277 m이고 10바퀴가 목표니까 22,770 m다. 그런데 시뮬레이터는 *****목표를 넘어서면 멈춘다*****는 규칙으로 끝나기 때문에 실제로는 **22,774.7 m**에서 멈춘다. 딱 **4.7 m** 초과다.

내 코드는 그 4.7 m 꼬리를 *****11번째 바퀴*****로 썼다. 그래서:

```
채점용 = "마지막 1바퀴" → 실제로는 마지막 4.7 m의 점 10개
```

5,265개여야 할 채점용 데이터가 10개였다. 게다가 하필 그 4.7 m 구간이 **직선**이라 잔차가 평소의 **100분의 1**($6.3e-5$ vs $1.08e-2$)이었다. 오차가 거의 없는 10개 점으로 채점하니 "예측 오차범위가 터무니없이 크다"는 결론이 나온 것이다.

비유 — 시험 범위 착각. "마지막 단원으로 시험 보자"고 했는데, 실수로 **마지막 페이지의 연습문제 3개만** 출제 한 것이다. 게다가 그 3개가 하필 제일 쉬운 문제였다. 학생이 다 맞혔다고 "이 학생은 실력에 비해 자신감이 없다"고 결론 내리면 완전히 틀린 것이다.

6-3. 어떻게 확인했나

추측하지 않고 재봤다. 같은 GP를 **학습 구간**에서 채점해 봤더니:

```
학습 구간에서: z_std = 0.993, 0.994 ← 완벽하게 정상
채점 구간에서: z_std = 0.062, 0.088 ← 이상
```

모델이 아니라 **채점 구간이 이상하다**는 게 확정됐다.

6-4. 고침 + 재발 방지

완주한 바퀴만 세고, 꼬리 조각은 버리도록 고쳤다. 그리고 회귀 테스트

`test_lap_block_split_ignores_partial_lap`을 넣었다. 이 테스트는 일부러 **"목표를 4.7 m 넘겨 끝난"** 가짜 데이터를 만들어서, 채점 구간이 제대로 한 바퀴 전체가 되는지 확인한다.

고친 뒤: **채점용 5,265개 = 정확히 한 바퀴.** 

7. 발견 3 — 가장 큰 것: AI가 학습을 포기하고 있었다

이번 작업에서 나온 가장 중요한 발견이다.

7-1. 이상 신호

수정 후 측정에서 이런 값이 보였다:

```
sigma_n (gamma 채널) = 1.0000 ← 정확히 1
sigma_f (gamma 채널) = 0.0010 ← 허용 최솟값
```

```
lengthscale = [0.001, 0.001, 0.001] ← 전부 허용 최소값
```

개념 6을 떠올려 보자. **sigma_f**는 **설명 가능한 신호**, **sigma_n**은 **설명 못 하는 잡음**이었다. 이 값들이 뜻하는 바는 이거다:

"이 데이터는 전부 무작위 잡음입니다. 저는 아무것도 설명할 수 없습니다."

GP가 학습을 포기한 것이다. 이 상태의 GP는 예측값이 항상 ~0이라, 잔차 보정을 사실상 안 하는 것과 같다.

게다가 여러 변형에서 이 값이 **소수점까지 똑같았다**. 우연이 아니라 **전부 같은 곳에 빠져 있다**는 뜻이다.

7-2. 물리인가 버그인가 — 갈림길

여기서 두 가지 해석이 가능했고, 이게 결론을 완전히 뒤집는다:

- **해석 A (물리):** 정말로 이 데이터로는 gamma를 예측할 수 없다. → 자연스러운 결과
- **해석 B (버그):** 예측할 수 있는데 **찾는 데 실패했다**. → 고쳐야 할 결함

추측으로 결론 내리지 않고 실험으로 갈랐다. 학습 시작점만 바꿔가며 다시 학습시켰다.

7-3. 결정적 실험

시작점 (init_lengthscale)	sigma_f	sigma_n	NLML	lengthscales
0.1	5.7140	0.4516	1376.5	[1.868, 0.914, 0.938]
0.3	5.7143	0.4516	1376.5	[1.868, 0.914, 0.938]
1.0 ← 기존 기본값	0.0010	1.0000	2837.9	[0.001, 0.001, 0.001]
3.0	0.0010	1.0000	2837.9	[0.001, 0.001, 0.001]
10.0	0.0432	0.9991	2837.9	[0.001, 0.001, 0.001]

NLML은 낮을수록 좋은 점수다(개념 6 참고).

- 시작점 0.1 또는 0.3 → **NLML 1376.5** (제대로 학습함)
- 시작점 1.0 (기존 기본값) → **NLML 2837.9** (포기함)

차이가 1461이다. 근소한 차이가 아니라 **완전히 다른 모델**이다. **해석 B(버그)**로 확정됐다.

7-4. 왜 이런 일이 — "국소최적"

국소최적(local optimum)이란?

비유 — 안개 낀 산에서 가장 낮은 곳 찾기. 당신은 안개 속 산에 있고, **가장 낮은 지점**을 찾아야 한다. 보이는 건 발밑뿐이다. 전략은 하나: **계속 내리막으로 걷는다**. 더 이상 내려갈 곳이 없으면 도착.

그런데 문제가 있다. 산에는 **작은 웅덩이**들이 있다. 웅덩이에 빠지면 사망이 다 오르막이라 *****여기가 제일 낮은 곳이다*****라고 착각하고 멈춘다. 실제로는 산 너머에 훨씬 깊은 계곡이 있는데도.

- **웅덩이 = 국소최적** (근처에서만 최선)
- **진짜 계곡 = 전역최적** (전체에서 최선)

어느 웅덩이에 빠지는지는 순전히 출발점에 달렸다.

우리 상황이 정확히 이거였다. 기존 코드는 **출발점을 딱 하나(1.0)만** 써봤고, 그게 하필 **"전부 잡음이다"**라는 **웅덩이**로 굴러떨어지는 자리였다.

왜 "전부 잡음"이 웅덩이가 되나? "설명 못 한다"고 선언하면 **틀려도 안 틀린 게 된다**. 오차범위를 최대한 벌려놓고 "다 이 안에 있습니다" 하면 되니까. 시험에서 모든 문제에 "모르겠음"이라고 쓰면 최소한 오답은 아닌 것과 비슷하다. 국소적으로는 나쁘지 않은 전략이라 빠져나오기가 어렵다.

7-5. 고침 — 다중 재시작

여러 출발점에서 각각 내려가 보고, 그중 제일 깊은 곳을 고르게 했다.

출발점 후보: {0.3, 1.0, 3.0} → NLML이 가장 낮은 결과 채택

비유 — 등산객 3명을 서로 다른 지점에서 출발시켜 각자 내려가게 한 뒤, 가장 낮은 곳에 도착한 사람의 결과를 쓰는 것.

이건 사실 **GP 분야의 표준 관행**이다. Type-II ML은 웅덩이가 많은 걸로 잘 알려져 있어서, 여러 번 시작하는 게 교과서적 처방이다. 이 프로젝트는 그걸 안 하고 있었다.

7-6. ★ 이건 3주 전부터 있던 문제였다

기록([handoff.md](#))을 다시 보니 **2026년 7월 30일에 이런 메모가 있었다:**

*"gamma 채널의 lengthscale이 하한($1e-3$) 근처다 — 그 차원 민감도가 매우 날카롭게 잡혔다는 뜻인데 **버그인** 지 데이터 특성인지 아직 판단하지 않았다."*

그게 바로 이 현상이었다. 그리고 7월 31일 기록에는 "해소됐다"고 적혀 있었는데, 실제로는 **고친 게 아니라 조건이 바뀌면서 출발점이 우연히 다른 골짜기에 떨어진 것**으로 보인다. (이건 추측이다 — 그때 설정으로 재시작 실험을 돌려야 확정된다.)

의미: 과거에 기록된 GP 성능 수치 중 상당수가 **GP가 부분적으로 포기한 상태에서 측정됐을 가능성**이 있다. 그래서 이 걸 미결 과제 #19로 등록했다.

교훈: "이상한데 판단 보류"라고 적어둔 메모는 **나중에 반드시 돌아가서 확인해야 한다**. 이번엔 3주 만에 돌아갔고, 그게 이번 작업에서 제일 큰 소득이었다.

8. 요청받은 실험 — 커널에 "과거"를 넣어보기

작업 중간에 *****커널 입력에 현재 상태뿐 아니라 이전 상태, 전전 상태도 넣어서 비교해 달라*****는 요청을 받았다.

8-1. 왜 이게 말이 되는 아이디어인가

기존 GP는 **지금 이 순간만** 본다:

z = [지금의 횡속도, 지금의 회전율, 직전 핸들각]

과거를 전혀 기억하지 않는다. 이걸 "기억이 없다(memoryless)"고 한다.

비유 — 기억상실증 의사. 환자를 볼 때마다 **처음 보는 것처럼** 진단하는 의사가 있다. "지금 열이 38도다"만 보고 판단한다. 그런데 실제로는 *****어제는 37도였고 오늘 38도로 오르는 중*****인지, *****어제 40도였다가 떨어지는 중*****인지에 따라 진단이 완전히 달라진다.

자동차 타이어도 마찬가지다. 타이어는 **직전까지 어떻게 눌리고 비틀렸는지에 따라** 지금 힘이 달라진다(이걸 타이어 이완 거동이라고 한다). 지금 상태만 봐서는 그걸 알 수 없다.

과거를 주면, 지금 "잡음"으로 버려지는 것 중 일부가 "설명 가능한 신호"로 바뀔 수 있다. 그러면 예측 오차범위가 실제에 가까워질 수 있다. 합리적인 가설이다.

8-2. 해석이 두 갈래였다

"과거를 넣어달라"는 요청에 두 가지 해석이 가능했고, 둘 다 만들어서 재봤다.

해석 A — full 모드: 상태와 핸들을 통째로 지연

$$z = [v_{y_k}, v_k, \delta_{\{k-1\}}, v_{y_{\{k-1\}}}, v_{\{k-1\}}, \delta_{\{k-2\}}, v_{y_{\{k-2\}}}, v_{\{k-2\}}, \delta_{\{k-3\}}]$$

└─── 지금 ──┘
└─── 한 스텝 전 ──┘
└─── 두 스텝 전 ──┘

→ 3개 시점 × 3개 항목 = **9차원**

해석 B — delta 모드: 핸들 이력만 추가

$$z = [v_{y_k}, v_k, \delta_{\{k-1\}}, \delta_{\{k-2\}}, \delta_{\{k-3\}}]$$

└─── 지금 상태 ──┘
└─── 핸들 이력 3개 ──┘

→ **5차원**

B가 요청하신 문구("delta_k, delta_{k-1}, delta_{k-2}")에 문자 그대로 가깝다. 차원도 훨씬 덜 늘어서(9 vs 5), **개념 5의 "차원의 저주" 관점에서 B가 유리하다.**

왜 인덱스가 한 칸 밀렸나: 4장에서 본 인과성 때문이다. δ_k 는 GP한테 물어보는 시점에 아직 존재하지 않는다. 그래서 쓸 수 있는 가장 최신 핸들각이 $\delta_{\{k-1\}}$ 이다. 항목 개수는 요청하신 3개와 같고, 시점만 한 칸씩 과거다.

8-3. 실전 배포도 같이 고쳐야 했다

과거를 쓰려면 **실제 주행 중에도 GP가 과거를 기억하고 있어야 한다.** 그래서 컨트롤러에 **이력 버퍼**(최근 몇 스텝을 담아두는 작은 저장소)를 달았다.

여기서 작은 문제가 하나 있었다. **주행 시작 직후에는 과거가 없다.** 어떻게 할까?

0으로 채우기로 했다. 이유가 있다 — 시뮬레이터는 **정지 상태(모든 값 0)에서 출발**하기 때문에, "없는 과거 = 0"이 **실제 초기 조건과 정확히 일치**한다. 학습과 실전이 똑같은 규칙을 쓰므로 4장의 불일치가 재발하지 않는다.

9. 숫자로 본 최종 결과

9-1. 개루프(모의고사) 결과 — 커널 입력 비교

랩 블록 분할 + 재시작 수정을 전부 적용한 뒤의 **공정한 비교**다.

변형	차원	NLPD	기준선 대비	z_std	95%커버	$\sigma_n(v_y)$	$\sigma_n(y)$
옛 규약 δ_k (배포 불가)	3	-6.326	-4.59%	1.119	94.7%	0.0461	0.1427
기준선 $\delta_{\{k-1\}}$	3	-6.048	—	0.982	95.3%	0.0583	0.4516
full lag1	6	-6.532	-8.00%	0.957	94.7%	0.0406	0.2375
full lag2	9	-6.749	-11.58%	1.006	93.8%	0.0310	0.1794
delta lag1	4	-6.057	-0.15%	0.983	95.3%	0.0576	0.4454
delta lag2	5	-6.059	-0.19%	0.982	95.2%	0.0573	0.4447

읽는 법 세 가지:

① **핸들 이력만 늘리는 건 효과가 없다.** -0.15%, -0.19%는 측정 잡음 수준이다. 차원을 4D, 5D로 늘렸는데 얻은 게 없다.

왜 그럴까 (추측 — 검증 안 됨): 이 시스템의 비용함수에 **핸들을 급격히 돌리면 벌점을 주는 항**(R_{ddelta})이 있다. 그래서 핸들이 아주 부드럽게 움직인다. 부드럽다는 건 $\delta_{\{k-2\}}$, $\delta_{\{k-3\}}$ 가 $\delta_{\{k-1\}}$ 과 거의 같은 값이라는 뜻이다. 같은 정보를 세 번 주는 셈이니 새로 얻는 게 없다.

② **효과는 "상태 이력"에서 나온다.** full lag1이 -8.00%, lag2가 -11.58%. v_y 와 y 의 과거가 타이어의 최근 이력을 담고 있고, 그게 원래 버려지던 정보라는 뜻이다.

메커니즘의 증거: σ_n (잡음 뒤편)이 계속 줄어든다.

γ 채널의 σ_n :
기준선 lag1 lag2
0.4516 → 0.2375 → 0.1794

"설명 못 하는 잡음"으로 분류되던 것이 "설명 가능한 신호"로 옮겨간 것이다. 이게 8-1에서 세운 가설이 맞았다는 직접 증거다.

③ **full lag는 "쓸 수 없는 옛 규약"보다도 좋다.** 배포 불가능한 δ_k 규약이 -4.59%인데, full lag2는 -11.58%다. 인과성을 지키느라 잃은 정보를, 상태 이력으로 되찾고도 남는다. 이번 비교에서 가장 쓸모 있는 결론이다.

9-2. 페루프(실전) 결과 — **lpf_ay4** (배운 코스)

케이스	RMS e_y (경로이탈)	z_std	95%커버	NLPD
보정 없음	1.3691e-2	—	—	—
칼만필터(KF)	8.6269e-3	0.831	96.1%	-4.64
GP — 수정 전	8.9134e-3	2.192	80.6%	-4.55
GP — 수정 후	8.9241e-3	1.153	94.3%	-5.87
GP — full lag2 (9D)	8.8992e-3	1.116	92.4%	-6.64

케이스	RMS e_y (경로이탈)	z_std	95%커버	NLPD
GP — delta lag2 (5D)	8.9245e-3	1.147	94.4%	-5.89

★ 이 표의 3번째와 4번째 줄이 이번 작업의 핵심 성과다.

z_std 2.192 → 1.153 (1.00이 완벽)
 95% 커버 80.6% → 94.3% (95%가 목표)

"자기가 말한 것보다 2배 넘게 틀리던" 상태에서 "말한 만큼 틀리는" 상태가 됐다. 그리고 이건 개루프 점수(0.982)와 거의 같다 — 즉 모의고사와 실전 점수가 드디어 일치한다. 3장에서 출발한 질문이 해결된 것이다.

그리고 이 개선은 커널을 바꿔서가 아니라 버그 2개를 고쳐서 나왔다.

9-3. 페루프 결과 — **rtf_ay6** (가장 어려운 낫선 상황)

케이스	RMS e_y	z_std	95%커버	NLPD
보정 없음	4.4078e-2	—	—	—
칼만필터(KF)	2.0330e-2	1.786	79.6%	-3.39
GP — 수정 전	3.2710e-2	2.839	62.9%	-1.11
GP — 수정 후	3.2915e-2	2.914	54.2%	-0.96
GP — full lag2 (9D)	3.8734e-2	2.380	61.2%	-1.84
GP — delta lag2 (5D)	3.3026e-2	2.904	53.7%	-0.99

여기는 안 고쳐졌다. 오히려 조금 나빠졌다. (62.9% → 54.2%)

그리고 **칼만 필터**가 모든 면에서 **GP**를 이긴다. 캘리브레이션(1.786 vs 2.914), 커버리지(79.6% vs 54.2%), 경로 추종 (2.03e-2 vs 3.29e-2) 전부.

full lag2의 트레이드오프: 캘리브레이션은 개선한다(z_std 2.914 → 2.380, 커버 54.2% → 61.2%). 그러나 **경로 이탈이 17.7% 악화된다**(3.29e-2 → 3.87e-2). 공짜가 아니다.

9-4. 비교가 공정한지 확인했다

GP만 고쳤으니 **칼만 필터 결과는 안 변해야 정상**이다. 확인차 다시 돌려봤다:

이전: z_std 0.831, 커버 96.1%, NLPD -4.64
 재실행: z_std 0.831, 커버 96.1%, NLPD -4.638

소수점까지 동일. 내 수정이 GP 경로만 건드렸다는 증거다. 이런 확인을 ****통제(control)****라고 한다.

비유 — 대조군. 신약 실험에서 약을 안 먹은 그룹의 상태가 안 변해야, 약 먹은 그룹의 변화를 약 덕분이라고 말할 수 있다. 대조군까지 같이 변했다면 실험 환경 자체가 오염된 것이다.

10. 지금 어디까지 왔나

✅ 해결된 것

항목	내용
모의고사 vs 실전 격차	원인 규명 + 해결. 커널이나 분포이동이 아니라 버그 2개 였다
배운 상황에서의 과신	해결. z_std 2.192 → 1.153, 커버 80.6% → 94.3%
GP vs KF (배운 상황)	GP가 역전. NLPD -5.87 vs -4.64. 커버리지도 GP(94.3%)가 목표 95%에 더 가깝다
학습 절차의 신뢰성	다중 재시작으로 "포기하는 GP" 문제 해결
측정 도구의 정당성	홀드아웃이 규칙 위반이던 것을 랩 블록 분할로 교체

❌ 안 고쳐진 것 (정직하게)

항목	내용
낮선 상황에서의 과신	rtf_ay6 에서 z_std 2.914, 커버 54.2%. 오히려 나빠졌다
낮선 상황에서 KF에 완패	캘리브레이션·추종 양쪽 다 진다
경로 추종 성능	배운 상황에서도 KF가 근소하게 앞선다 ($8.63e-3$ vs $8.92e-3$)
과거 수치의 신뢰성	7장 문제가 언제부터였는지 모른다. 과거 표들을 재검증해야 함

✦ 결론적으로

"배운 것에 대해서는 GP가 자기 확신을 정직하게 말한다. 처음 보는 것에 대해서는 아직 아니다."

이게 지금 위치다. 논문의 주장 범위를 이 선에 맞춰야 한다.

11. 앞으로 갈 방향

11-1. 최우선 — 낮선 상황(OOD)에서의 과신 (미결 #17)

이게 남은 유일한 큰 문제다. 원인 후보가 두 개 있고, 둘 다 커널의 구조와 관련 있다. (처음에 "커널을 바꿔야 할 것 같다"고 하신 직관이 **여기서 다시 유효해진다**. 다만 차원을 늘리는 쪽이 아니라 **잡음 모델** 쪽이다.)

후보 A — 정상(stationary) 커널의 구조적 상한

지금 커널은 어디서나 똑같은 성질을 가정한다. 그래서 예측 오차범위가 $\sigma_f^2 + \sigma_n^2$ 라는 **천장**을 못 넘는다.

비유 — 눈금이 30cm까지밖에 없는 자. 아무리 긴 물건을 재도 "30cm 이상"이라고밖에 못 말한다. 지금 GP는 아무리 낮선 상황이어도 "학습 때 봤던 산포 정도" 이상으로는 불확실하다고 말할 수 없다.

후보 B — 등분산(homoscedastic) 가정 ← 더 직접적인 후보

지금은 잡음 크기 σ_n 이 **딱 하나**다. 살살 도는 상황이든 험하게 도는 상황이든 **같은 오차범위**를 쓴다.

비유 — 계절 구분 없는 일기예보. "우리 지역 기온 오차는 항상 ± 2 도입니다"라고 말하는 예보관이 있다고 하자. 봄가을엔 맞을지 몰라도, 날씨가 요동치는 환절기엔 ± 8 도씩 틀린다. 그런데도 계속 ± 2 도라고 말한다. **틀리는 정**

도가 상황에 따라 다르다는 걸 표현할 수단이 아예 없기 때문이다.

우리 GP가 정확히 이 상태다. $a_y=6$ (힘한 코너링)에서는 잔차 산포가 훨씬 커지는데, "여긴 더 많이 튼다"고 말할 수단이 없다.

해결 방향: σ_n 을 상수가 아니라 **상황에 따라 변하는 함수 $\sigma_n(z)$ **로 바꾼다. 이것 **이분산(heteroscedastic) 모델**이라고 한다.

좋은 소식 — 생각보다 싸다. 이 프로젝트는 **예측 오차범위를 제어에 쓰지 않는다** (평균값만 쓴다). 오차범위는 기록·평가용이다. 그래서 잡음 모델을 바꿔도 **MPC 최적화 코드는 손덜 필요가 없다**. 학습·기록 경로만 바뀐다.

11-2. 그 다음 — 과거 수치 재검증 (미결 #19)

7장의 "학습 포기" 문제가 **언제부터 있었는지 모른다**. 과거 기록의 GP 성능 수치 상당수가 그 상태에서 측정됐을 수 있다.

할 일: 4개 시나리오 전체를 수정본으로 다시 돌려 표를 갱신한다. 지금 갱신된 건 `lpf_ay4`와 `rtf_ay6` 둘뿐이다.

왜 중요한가: 논문에 쓸 표가 **부분적으로 고장 난 GP로 켜 숫자**라면, 결론 자체가 흔들린다. 이건 성능 문제가 아니라 **신뢰성 문제**다.

11-3. 결정해야 할 것 — 상태 이력을 채택할까 (미결 #18)

	얻는 것	잃는 것
full lag2 (9D)	개루프 NLPD -11.6%, OOD 캘리브 개선(2.914→2.380)	경로 추종 +17.7% 악화 , 차원 3배
delta lag2 (5D)	거의 없음 (-0.19%)	차원 증가만

지금 판단: 기본값은 **3차원 유지**. 이유는 두 가지다.

1. **순수 이득이 아니다.** 추종 성능을 대가로 받는다.
2. **프로젝트의 핵심 주장과 충돌한다.** 차원을 늘리면 "데이터 없는 곳에서 GP가 자신 없어 한다"는 논증이 약해진다 (개념 5의 차원의 저주).

단, 11-1의 이분산 모델을 넣은 뒤에는 다시 볼 가치가 있다. 그때는 추종 악화 없이 캘리브레이션만 얻을 수도 있다. 비교용 설정 파일 20개를 전부 만들어 뒀으므로 **언제든 한 줄로 다시 켤 수 있다**.

11-4. 그 후 — 논문 논증 재설계 (미결 #15)

결과가 이렇게 나왔으니 주장의 범위를 다시 잡아야 한다.

- "GP가 KF보다 낫다" → 너무 넓다. 낫선 상황에서는 사실이 아니다.
- "배운 범위 안에서는 GP의 불확실성 표현이 더 정직하다. 범위 밖에서는 아직 미해결이고, 그 원인은 등분산 가정으로 특정된다." → 측정으로 뒷받침되는 주장.

솔직하게 좁힌 주장이 **부풀린 주장보다 강하다**. 근거가 있기 때문이다.

11-5. 더 나중 — Part 2

Part 1(GP vs KF 비교)이 정리되면 Part 2로 간다: GP를 **실시간으로** 굴리는 문제. ① 미리 학습(현재) → ② 주행 중 학습 → ③ 회소 근사 → ④ 최근 데이터만 유지.

12. 이번 일에서 배운 것 (방법론)

기술적 결론과 별개로, **일하는 방식**에 대해 남길 게 있다.

교훈 1 — 이상한 결과는 모델이 아니라 측정 도구를 먼저 의심하라

6장에서 `z_std`가 0.04로 나왔을 때, "GP가 과소신뢰한다"고 결론 내릴 수도 있었다. 그러면 **완전히 틀린 방향으로 갔을 것이다**.

여러 변형이 한꺼번에 같은 극단값을 보이면 측정 장치 문제일 가능성이 높다. 실제로 채점용 데이터가 5,265개가 아니라 10개였다.

일반화: 실험 결과가 "너무 좋거나" "너무 이상하면", 결과를 해석하기 전에 **측정 방법부터 검산하라**.

교훈 2 — "판단 보류"라고 적은 메모는 반드시 돌아가라

7장의 문제는 **3주 전에 이미 관찰됐고 "버그인지 데이터 특성인지 미판단"**으로 기록돼 있었다. 그리고 그 사이의 모든 실험이 그 상태에서 이뤄졌다.

일반화: "나중에 보자"고 미룬 이상 신호는 **이자가 붙는다**. 미룬 기간 동안 나온 결과가 전부 재검증 대상이 되기 때문이다.

교훈 3 — 버그는 종종 "그럴듯한 결과"로 위장한다

4장의 시점 불일치는 **에러 메시지를 내지 않았다**. 프로그램은 잘 돌았고, 숫자도 그럴듯했다. 다만 **GP가 과신하는 것처럼 보이게 만들었다**.

7장의 학습 포기도 마찬가지다. "이 데이터는 예측 불가능하다"는 것도 **하나의 결론**처럼 보인다. 실제로는 최적화 실패였다.

일반화: **조용히 틀리는 버그가 제일 위험하다**. 그래서 이번에 두 버그 모두 **회귀 테스트**로 못 박았다. 사람의 주의력이 아니라 자동 검사에 맡긴다.

교훈 4 — 통제된 비교만이 인과를 말한다

이번 실험은 **한 번에 하나씩만** 바꿨다. 6개 커널 변형은 `n_lags`와 `lag_mode` 외에 모든 것(딕셔너리 크기 `M`, 초기값, 학습 데이터, 시드)이 동일하다. 그리고 칼만 필터를 재실행해 **안 변했음**을 확인했다.

일반화: 두 개를 동시에 바꾸고 결과가 좋아지면, **어느 것 덕분인지 영원히 알 수 없다**. 시간이 더 걸려도 하나씩 바꾸는 게 결국 빠르다.

교훈 5 — 캐시는 조용히 거짓말한다

이번에 발견한 문제 하나 더: 학습 결과를 재사용하는 **캐시**가 설정만 보고 있었다. **코드를 바꿔도 캐시는 "설정이 같으니 예전 걸 쓰자"**고 판단했다.

비유 — 레시피 번호로 도시락을 찾아 쓰는데, 레시피 내용이 바뀌어도 번호는 그대로라 **옛날 도시락**을 꺼내 먹는 상황.

코드 버전을 캐시 키에 넣어 고쳤다. 안 고쳤다면 이번 수정이 **적용되지 않은 채 "고쳤는데 효과가 없네"**라는 잘못된 결론이 나왔을 것이다.

13. 용어 사전

자동차·제어

용어	뜻	비유
모델(model)	실제 시스템을 수식으로 단순화한 것	지도 — 실제 땅은 아니지만 길 찾기엔 충분
선형 자전거 모델	4바퀴 차를 앞뒤 2바퀴로 단순화한 표준 모델	자동차를 자전거로 보기
선형(linear)	입력 2배 → 출력 정확히 2배	살살 당긴 고무줄
MPC	앞으로 여러 스텝을 계산해 놓고 첫 스텝만 실행, 매번 반복	체스 — 몇 수 앞 보고 지금 한 수
잔차(residual)	모델 예측과 실제의 차이	일기예보 오차 (25도 예보, 28도 실제 → +3도)
칼만 필터(KF)	시간 흐름을 따라 오차를 추정·보정하는 고전 기법	최근 3번 늦은 친구는 오늘도 늦겠지
RMS e _y	경로에서 벗어난 거리의 평균적 크기	작을수록 잘 따라감
a _y	코너링 가속도. 클수록 빠르고 험하게 돈다	4 = 보통, 6 = 험하게

머신러닝·통계

용어	뜻	비유
가우시안 프로세스 (GP)	예측값과 자신감 을 함께 내놓는 학습 방법	경력 30년 정비사 — "확실합니다" / "처음 봐서 모르겠는데요"
커널(kernel)	"두 상황이 비슷한가"를 재는 기준	닮음을 재는 자
차원(dimension)	GP가 보는 항목의 개수	3차원 = 항목 3개
차원의 저주	차원이 늘면 같은 데이터가 급격히 특성 해집	선 → 정사각형 → 정육면체에 점 10개 뿌리기
하이퍼파라미터	데이터를 보고 자동으로 정해지는 설정 값	—
lengthscale	얼마나 떨어지면 다른 상황으로 보는지	시야 범위 (근시안 vs 원시안)
sigma _f	설명 가능한 신호의 크기	목소리 크기
sigma _n	설명 불가능한 잡음의 크기	주변 소음
Type-II ML	데이터가 나올 확률이 최대가 되는 설정 값을 찾는 방법	발자국으로 범인 키 추정
NLML	Type-II ML의 점수. 낮을수록 좋음	골프 스코어
국소최적(local optimum)	근처에서만 최선인 지점. 전역최적이 따로 있음	안개 산의 작은 웅덩이

용어	뜻	비유
다중 재시작	여러 출발점에서 찾아보고 제일 좋은 걸 채택	등산객 3명 각자 내려보내기
홀드아웃(holdout)	학습에서 빼놓고 채점용으로만 쓰는 데이터	문제집 뒷장 안 보고 남겨두기
in-distribution	학습 때 본 것과 비슷한 상황	문제집에 있던 유형
OOD	학습 때 못 본 낯선 상황	처음 보는 유형
등분산 / 이분산	잡음 크기가 어디서나 같다 / 상황마다 다르다	계절 무시 예보 / 계절 반영 예보
정상(stationary) 커널	위치와 무관하게 성질이 같은 커널	눈금 상한이 정해진 자

불확실성 평가

용어	뜻	좋은 값
캘리브레이션	자신감이 정직한가	—
z_std	실제 오차 ÷ 예측 오차범위	1.0 (>1이면 과신)
95% 커버	95% 구간에 실제로 들어간 비율	95%
NLPD	예측 분포 전체의 품질 점수	낮을수록 좋음
과신(overconfident)	실제보다 자신만만	위험
과소신뢰	실제보다 겁먹음	덜 위험하지만 비효율

개발

용어	뜻	비유
개루프 / 페루프	예측이 결과에 영향 없음 / 영향 있음	축구 해설 / 축구 선수
커밋(commit)	코드의 저장 지점	게임 세이브 포인트
회귀 테스트	고친 버그가 되살아나는 걸 막는 자동 검사	문단속 확인 알람
캐시(cache)	계산 결과를 저장해 재사용하는 것	만들어 둔 도시락
인과성(causality)	결과 전에 원인을 알 수 없다는 시간 순서	로또 번호 보고 로또 살 수 없다
통제(control)	비교 시 다른 조건을 전부 동일하게 유지	신약 실험의 대조군

부록 — 이번 작업의 커밋 목록

커밋	내용
08c899f	칼만 필터 재튜닝 + NIS 진단 인프라

커밋	내용
6631535	페루프 N바퀴 주행 기능, GP 학습 출처 일원화
ad2806f	GP 예측분산 버그 수정 + M=2000
4d7c816	4케이스 실험 체계 신설
41fdd2c	7/30~31 기록
b20f5a9	학습/배포 특징 정합(delta 시점) + Type-II ML 다중 재시작
f0638d6	커널 입력 시간지연 축 + 랩 블록 홀드아웃
4753612	8/1 기록, 미결 #16 종결 / #17~19 신설

테스트 86개 전부 통과 (기존 77 + 이번 신규 9).

상세한 기술 기록과 미결 사항의 공식 목록은 [handoff.md](#)에 있다. 이 문서와 숫자가 어긋나면 [handoff.md](#)가 기준이다.